

Experiment 'p_6_2_f_move_even_to_front_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_6_2_f_move_even_to_front_array

Natural language statement: null

Method signature: p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)

Ensures

- `result[..] == evens(inputs[..]) + odds(inputs[..])`

Functional Code Given

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)`

The method should respect the following contract:

`ensures result[..] == evens(inputs[..]) + odds(inputs[..])`

The contract uses the following dafny code:

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}
```

```
function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNunqFmNYjEjbXvJfgotDoKCrusB

Model: gpt-5.5-2026-04-23

Created at: 1780662373

Finish reason: stop

Usage: 560 tokens in, and 8385 tokens out

```
{
```

```

var i := inputs.Length;
var es: seq<int> := [];
var os: seq<int> := [];

while i > 0
  invariant 0 <= i <= inputs.Length
  invariant es == evens(inputs[i..])
  invariant os == odds(inputs[i..])
  decreases i
{
  i := i - 1;
  assert 0 <= i < inputs.Length;
  assert es == evens(inputs[i + 1..]);
  assert os == odds(inputs[i + 1..]);
  assert |inputs[i..]| > 0;
  assert inputs[i..][0] == inputs[i];
  assert inputs[i..][1..] == inputs[i + 1..];

  if inputs[i] % 2 == 0 {
    assert inputs[i..][0] % 2 == 0;
    assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
    assert odds(inputs[i..]) == odds(inputs[i + 1..]);
    es := [inputs[i]] + es;
    assert es == evens(inputs[i..]);
    assert os == odds(inputs[i..]);
  } else {
    assert inputs[i..][0] % 2 != 0;
    assert evens(inputs[i..]) == evens(inputs[i + 1..]);
    assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i + 1..]);
    os := [inputs[i]] + os;
    assert es == evens(inputs[i..]);
    assert os == odds(inputs[i..]);
  }
}

var out := es + os;
assert i == 0;
assert inputs[i..] == inputs[..];
assert out == evens(inputs[..]) + odds(inputs[..]);

result := new int[|out|](j => out[j]);
assert |result[..]| == |out|;
assert forall j :: 0 <= j < |out| ==> result[..][j] == out[j];
assert result[..] == out;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_6_2_f_move_even_to_front_array/program.dfy(60,35): Error: index out of
  ↪ range
  |
60 | result := new int[|out|](j => out[j]);
  | ^

```

Dafny program verifier finished with 3 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnNy50mADxOtfAxpN7NunCOzLsr2f

Model: gpt-5.5-2026-04-23

Created at: 1780662577

Finish reason: stop

Usage: 1084 tokens in, and 1274 tokens out

```

{
  var i := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
    decreases i
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert es == evens(inputs[i + 1..]);
    assert os == odds(inputs[i + 1..]);
    assert |inputs[i..]| > 0;
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert inputs[i..][0] % 2 == 0;
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      assert odds(inputs[i..]) == odds(inputs[i + 1..]);
      es := [inputs[i]] + es;
      assert es == evens(inputs[i..]);
      assert os == odds(inputs[i..]);
    } else {
      assert inputs[i..][0] % 2 != 0;
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
    }
  }
}

```

```

    assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i + 1..]);
    os := [inputs[i]] + os;
    assert es == evens(inputs[i..]);
    assert os == odds(inputs[i..]);
  }
}

var out := es + os;
assert i == 0;
assert inputs[i..] == inputs[..];
assert out == evens(inputs[..]) + odds(inputs[..]);

result := new int[|out|](j => if 0 <= j < |out| then out[j] else 0);
assert |result[..]| == |out|;
assert forall j :: 0 <= j < |out| ==> result[..][j] == out[j];
assert result[..] == out;
}

```

Final Program

```

function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
    then [s[0]] + evens(s[1..])
    else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
    then [s[0]] + odds(s[1..])
    else odds(s[1..])
}

method p_6_2_f_move_even_to_front_array(inputs: array<int>) returns (result: array<int>)
  ensures result[..] == evens(inputs[..]) + odds(inputs[..])
{
  var i := inputs.Length;
  var es: seq<int> := [];
  var os: seq<int> := [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant es == evens(inputs[i..])
    invariant os == odds(inputs[i..])
    decreases i
  {
    i := i - 1;
  }
}

```

```

    assert 0 <= i < inputs.Length;
    assert es == evens(inputs[i + 1..]);
    assert os == odds(inputs[i + 1..]);
    assert |inputs[i..]| > 0;
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
        assert inputs[i..][0] % 2 == 0;
        assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
        assert odds(inputs[i..]) == odds(inputs[i + 1..]);
        es := [inputs[i]] + es;
        assert es == evens(inputs[i..]);
        assert os == odds(inputs[i..]);
    } else {
        assert inputs[i..][0] % 2 != 0;
        assert evens(inputs[i..]) == evens(inputs[i + 1..]);
        assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i + 1..]);
        os := [inputs[i]] + os;
        assert es == evens(inputs[i..]);
        assert os == odds(inputs[i..]);
    }
}

var out := es + os;
assert i == 0;
assert inputs[i..] == inputs[..];
assert out == evens(inputs[..]) + odds(inputs[..]);

result := new int[|out|](j => if 0 <= j < |out| then out[j] else 0);
assert |result[..]| == |out|;
assert forall j :: 0 <= j < |out| ==> result[..][j] == out[j];
assert result[..] == out;
}

```

Total Token Usage

Input tokens: 1644

Output tokens: 9659

Reasoning tokens: 8523

Sum of 'total tokens': 11303

Experiment Timings

Overall Experiment started at 1780662371617, ended at 1780662633357, lasting 261740ms (261.74 seconds)

Iteration #1 started at 1780662371618, ended at 1780662576468, lasting 204850ms (204.85 seconds)

Iteration #2 started at 1780662576570, ended at 1780662633337, lasting 56767ms (56.77 seconds)